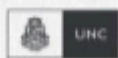


CERTIFICACIÓN UNIVERSITARIA



Manual para el alumno

Linux II



**WE WANT
SKILLS!**

mundosE

Manual para el Alumno	
Objetivo del encuentro.....	3
Contenido a desarrollar.....	3
Gestión de paquetes y actualizaciones en las distintas distribuciones.....	4
Introducción a Shell Scripting: definición, ventajas, estructura básica y estructuras comunes.....	7
Bibliografía.....	11

Este manual es un complemento de los encuentros sincrónicos de cada módulo. En él encontraremos un resumen de cada tema tratado en el encuentro, así como también recursos adicionales para poder profundizar sobre los conceptos vistos.

Manual para el Alumno: Linux II

Objetivo del encuentro:

El objetivo de este encuentro es que comprendas cómo administrar paquetes y actualizaciones en Linux, explorando las herramientas específicas de cada distribución como APT, YUM, DNF, Snap y Flatpak. Además, vas a aprender como crear scripts utilizando Bash, con el fin de automatizar tareas repetitivas, gestionar sistemas de forma eficiente y personalizar flujos de trabajo según necesidades particulares.

Al finalizar el encuentro, con los laboratorios y un poco de práctica, vas a estar capacitado para implementar soluciones prácticas en la gestión de sistemas Linux, optimizando procesos y reduciendo tiempos mediante herramientas automatizadas y personalizadas para tareas específicas. Esta habilidad, entre otras cosas, te permitirá mantener sistemas actualizados, mejorar su seguridad y operar de manera profesional en entornos Linux modernos.

Contenido a desarrollar:

- ✓ Gestión de paquetes y actualizaciones en las distintas distribuciones.
- ✓ Introducción a Shell Scripting: definición, ventajas, estructura básica y estructuras comunes.

1. Gestión de paquetes y actualizaciones en las distintas distribuciones.

¿Qué es un gestor de paquetes?

Un gestor de paquetes es una herramienta fundamental que permite a los usuarios instalar, actualizar, configurar y eliminar software de manera centralizada en un sistema Linux. Estas herramientas son esenciales para garantizar que el sistema operativo y sus aplicaciones funcionen de manera segura, estable y eficiente. Además, simplifican la tarea de resolver dependencias, evitando conflictos entre paquetes o bibliotecas necesarias para ejecutar programas.

Principales gestores de paquetes

- **APT (Advanced Package Tool):** Utilizado en distribuciones basadas en Debian y Ubuntu. Es reconocido por su simplicidad y eficiencia.
- **YUM/DNF:** Predominantes en Fedora, CentOS y RHEL. DNF, en particular, ofrece mejoras significativas en la velocidad y el manejo de dependencias.
- **Zypper:** Exclusivo de openSUSE, con un enfoque en la flexibilidad y capacidades avanzadas de gestión.
- **Snap y Flatpak:** Diseñados para aplicaciones modernas, ofrecen compatibilidad con múltiples distribuciones y son ideales para software aislado del sistema base.

Ejemplos prácticos con APT

1. **Actualizar la lista de paquetes disponibles:**

```
sudo apt update
```

Este comando consulta los repositorios configurados para buscar actualizaciones.

2. **Instalar un paquete:**

```
sudo apt install nombre_paquete
```

Descarga e instala el paquete indicado junto con sus dependencias.

3. **Actualizar el sistema completo:**

```
sudo apt upgrade
```

Instala todas las actualizaciones disponibles para los paquetes instalados.

4. **Eliminar un paquete:**

```
sudo apt remove nombre_paquete
```

Elimina el paquete instalado, pero conserva sus archivos de configuración.

5. **Eliminar completamente un paquete y sus configuraciones:**

```
sudo apt purge nombre_paquete
```

Ejemplos prácticos con YUM/DNF

1. **Instalar un paquete:**

```
sudo dnf install nombre_paquete
```

Este comando descarga e instala el paquete junto con sus dependencias.

2. **Actualizar paquetes existentes:**

```
sudo dnf update
```

Actualiza todos los paquetes instalados en el sistema.

3. **Eliminar un paquete:**

```
sudo dnf remove nombre_paquete
```

Elimina un paquete y sus dependencias no utilizadas.

Ejemplos prácticos con Zypper

1. **Actualizar repositorios:**

```
sudo zypper refresh
```

Sincroniza los repositorios configurados con las últimas versiones

disponibles.

2. **Instalar un paquete:**

```
sudo zypper install nombre_paquete
```

Descarga e instala el paquete indicado.

3. **Eliminar un paquete:**

```
sudo zypper remove nombre_paquete
```

Elimina el paquete especificado y sus dependencias.

Ejemplos prácticos con Snap

1. **Instalar una aplicación:**

```
sudo snap install nombre_aplicacion
```

Instala la aplicación desde el repositorio oficial de Snap.

2. **Actualizar aplicaciones instaladas:**

```
sudo snap refresh
```

Actualiza todas las aplicaciones Snap instaladas en el sistema.

3. **Eliminar una aplicación:**

```
sudo snap remove nombre_aplicacion
```

Desinstala la aplicación.

Ejemplos prácticos con Flatpak

1. **Agregar un repositorio remoto:**

```
flatpak remote-add --if-not-exists flathub
```

Repositorio oficial: <https://flathub.org/repo/flathub.flatpakrepo>

Configura Flathub como un repositorio remoto para instalar aplicaciones.

2. Instalar una aplicación:

```
flatpak install flathub nombre_aplicacion
```

Descarga e instala la aplicación desde el repositorio Flathub.

3. Ejecutar una aplicación instalada:

```
flatpak run nombre_aplicacion
```

Ejecuta la aplicación instalada usando Flatpak.

4. Eliminar una aplicación:

```
flatpak uninstall nombre_aplicacion
```

Elimina la aplicación del sistema.

Recursos para profundizar:

- [Documentación oficial de APT](#)
- [Guía de Snap](#)
- [Introducción a Flatpak](#)

2. Introducción a Shell Scripting: definición, ventajas, estructura básica y estructuras comunes.

Definición

Un shell script es un archivo de texto que contiene una serie de comandos que se ejecutan secuencialmente en un shell. Permite automatizar tareas repetitivas y ejecutar procesos complejos de forma eficiente. Esto lo convierte en una herramienta poderosa para administradores de sistemas y desarrolladores que buscan optimizar flujos de trabajo (pipelines)

Ventajas

- **Automatización de tareas repetitivas:** Elimina la necesidad de realizar manualmente acciones que se repiten con frecuencia.
- **Ahorro de tiempo:** Ejecutar varios comandos con un solo archivo ahorra minutos valiosos en operaciones rutinarias.
- **Personalización:** Permite crear scripts adaptados a necesidades específicas, por ejemplo: configurar entornos personalizados para nuestra organización.

- **Facilidad de administración:** Es muy útil para tareas de administración de servidores, como por ejemplo la creación de backups, la gestión de usuarios y el monitoreo de sistemas, entre otras

Estructura básica

Un script básico incluye:

- **Shebang:** Indica el intérprete que ejecutará el script.

```
1 #!/bin/bash
```

- **Comandos:** Instrucciones ejecutadas en orden.

```
1 echo "Hola, este es un script básico"
```

- **Comentarios:** Empiezan con **#** y no se ejecutan.

```
1 # Este es un comentario explicativo
```

Estructuras comunes

Condicionales:

```
if [ $1 -gt 10 ]; then
    echo "Mayor que 10"
else
    echo "Menor o igual a 10"
fi
```

- **Bucles:**

For:

```
for i in {1..5}; do
    echo $i
done
```


While:

```
contador=1
while [ $contador -le 5 ]; do
    echo $contador
    ((contador++))
done
```

Funciones:

```
funcion_ejemplo() {
    echo "Esto es una función"
}
funcion_ejemplo
```

Ejemplos adicionales

Les proponemos que puedan crear los siguientes script para profundizar en los conocimientos adquiridos sobre Shell Scripting.

Script para saludar según la hora del día:

```
#!/bin/bash
hora=$(date +%H)
if [ $hora -lt 12 ]; then
    echo "Buenos días"
elif [ $hora -lt 18 ]; then
    echo "Buenas tardes"
else
    echo "Buenas noches"
fi
```

Explicación:

- **date +%H**: Obtiene la hora actual del sistema en formato de 24 horas.
- **-lt**: Operador de comparación en Bash que significa "menor que".
- Según el valor obtenido, se compara con 12 y 18 para determinar si es mañana, tarde o noche.

Generador de backups simples:

```
1 #!/bin/bash
2 echo "Creando un backup de la carpeta /home/usuario/documentos..."
3 tar -czvf backup_$(date +%F).tar.gz /home/usuario/documentos
4 echo "Backup completado."
```

Explicación:

- `tar -czvf`: Comando para crear un archivo comprimido en formato `.tar.gz`.
- `$(date +%F)`: Agrega la fecha actual en formato `YYYY-MM-DD` al nombre del backup.
- Esto ayuda a mantener copias de seguridad organizadas por fecha.

Contador de archivos en un directorio:

```
1 #!/bin/bash
2 dir=$1
3 if [ -d "$dir" ]; then
4     echo "El directorio $dir contiene $(ls -l "$dir" | wc -l) archivos."
5 else
6     echo "El directorio $dir no existe."
7 fi
```

Explicación:

- `$1`: Representa el primer argumento que se pasa al script, en este caso, el nombre del directorio.
- `-d "$dir"`: Comprueba si la ruta ingresada es un directorio válido.
- `ls -l "$dir" | wc -l`: Lista los archivos en el directorio y los cuenta.
- Si el directorio existe, imprime la cantidad de archivos que contiene. Si no, muestra un mensaje de error.

Script interactivo para configurar un usuario:

```
1 #!/bin/bash
2 echo "Ingrese el nombre de usuario a crear:"
3 read usuario
4 sudo adduser $usuario
5 echo "Usuario $usuario creado exitosamente."
```

Explicación:

- `read usuario`: Captura la entrada del usuario y la almacena en la variable `usuario`.
- `sudo adduser $usuario`: Crea un nuevo usuario en el sistema con el nombre ingresado.
- Luego, muestra un mensaje de confirmación.

Monitoreo básico de uso de disco:

```
1 #!/bin/bash
2 echo "Uso de disco en el sistema:"
3 df -h
```

Explicación:

- `df -h`: Muestra el uso del disco en formato legible para humanos.
- Es útil para monitorear el espacio disponible en cada partición del sistema.

Recursos para profundizar:

- [Guía de scripting en Bash](#)
- [¿Qué es Bash y para que sirven los Bash Scripts?](#)

Bibliografía

- Morales, Daniel. (2020). *Linux para principiantes: Guía práctica para aprender el sistema operativo más popular*.
- Garrido, José María. (2019). *Comandos Linux y Shell Scripting: Manual Práctico*.
- Pérez, Juan. (2018). *Administración de sistemas Linux: Teoría y práctica*.
- [Documentación oficial de GNU/Linux](#)